

IA Aplicada com LLMs

Aula 10 — Escrever um servidor

A ferramenta vira servidor

```
from mcp.server.fastmcp import FastMCP

servidor = FastMCP("politica-de-despesas")

@servidor.tool()
def consultar_politica(categoria: str) -> dict:
    """Devolve o teto de reembolso vigente para uma categoria de despesa."""
    return POLITICA[categoria]
```

A *docstring* vira a descrição. As anotações de tipo viram o `inputSchema`.

O corpo é o mesmo da Aula 05 — e isso é intencional: MCP não pede reescrita de lógica. Pede que a **fronteira seja declarada**.

Parte 1

A descrição é prompt

E agora é prompt de terceiros

A Aula 03 (nota 04) estabeleceu que a descrição é o texto pelo qual o modelo decide **se** e **como** chamar.

Num servidor MCP, três propriedades novas se somam:

1. **o consumidor é desconhecido** — outros modelos, outros prompts de sistema, outros objetivos. Suposições implícitas deixam de valer;
2. **o contrato é público** — renomear um parâmetro quebra todo cliente, e nenhum compilador avisa;
3. **a mudança é silenciosa** — o cliente que passa a chamar errado não lança exceção. Produz resultado **pior**.

A regra de escrita para descrição publicada

```
""""Devolve o teto de reembolso vigente para uma categoria de despesa.
```

```
categoria: um de "refeicao", "transporte", "hospedagem", "outros".  
Valores fora dessa lista devolvem erro com a lista válida.
```

```
NÃO decide se uma despesa específica é aprovada, e NÃO considera  
exceções de viagem internacional – para isso, leia o recurso  
regulamento://despesas/art-19.  
""""
```

O que faz · o formato exato dos argumentos · o que NÃO faz.

A última parte é a mais esquecida, e a que mais evita chamada indevida.

Duas descrições, e a diferença medida

```
DESCRIÇÃO A    "Consulta a política."  
-> categoria="almoço"      (não existe)  
-> sem argumento          (schema rejeita)  
-> 3 chamadas até acertar
```

```
DESCRIÇÃO B    "...categoria: um de "refeicao", "transporte",  
                "hospedagem", "outros"."  
-> categoria="refeicao"  
-> 1 chamada
```

O ganho não é de elegância. São **duas chamadas a menos por execução**, em todo cliente do servidor, para sempre.

Parte 2

Recurso, erro e escrita

Recurso e ferramenta, no mesmo domínio

```
@servidor.resource("regulamento://despesas/{artigo}")
def artigo(artigo: str) -> str:
    """Texto integral de um artigo do regulamento."""
    return REGULAMENTO[artigo]

@servidor.tool()
def registrar_parecer(despesa: str, veredito: str, chave: str) -> dict:
    """Registra o parecer. Idempotente pela chave."""
```

O **regulamento** é **recurso** porque a aplicação sabe, antes do laço começar, que o agente vai precisar dele.

Deixá-lo como **ferramenta** transferiria ao modelo uma decisão que não é dele.

A idempotência atravessou a fronteira

```
if (existente := PARECERES.get(chave)):  
    return {**existente, "ja_existia": True}
```

Na Aula 05, a chave protegia contra *retry*, checkpoint e laço detectado — todos **dentro do mesmo processo**.

Aqui ela protege contra algo novo: **clientes que o autor do servidor não controla**.

Um agente mal escrito do outro lado da conexão vai repetir chamadas. A chave é o que impede que a repetição vire dano.

O erro tem duas naturezas — e confundi-las é o defeito comum

Natureza	Exemplo	Quem trata
de protocolo	método inexistente, JSON malformado	o cliente, como falha de transporte
de domínio	categoria inválida, id inexistente	o modelo , como observação

erro de domínio devolvido como falha de PROTOCOLO

→ SOME do contexto. O modelo nunca sabe que errou, e repete.

erro de infraestrutura devolvido como OBSERVAÇÃO

→ quatro reformulações educadas até o orçamento acabar.

O erro de domínio volta como conteúdo

```
if categoria not in POLITICA:
    return {"erro": "categoria desconhecida",
            "recebido": categoria,
            "validos": sorted(POLITICA),
            "sugestao": "chame novamente com um dos valores de 'validos'"}

```

Responde às três perguntas da Aula 05: **o que errou · qual era o certo · o que fazer agora.**

Versionamento de um contrato público

O carimbo da Aula 03 ganha dois campos: **revisão da especificação** e **versão do servidor**.

E há uma diferença de grau que vale nomear:

Nos casos anteriores, quem muda o carimbo é quem roda o experimento. Num servidor de terceiro, **quem muda é outra pessoa** — sem aviso, e sem que o resultado deixe de ser sintaticamente válido.

Fixar a versão · tratar a descrição como código sob revisão · não remover nem renomear parâmetro publicado.

O que fica

- Vinte e duas linhas transformam a ferramenta da Aula 05 em serviço
- A descrição é **prompt entregue a um consumidor desconhecido**
- Recurso × ferramenta é decidido por **quem controla a inclusão**
- Idempotência protege contra **o cliente alheio**

Próxima nota: o que o protocolo **cobra** — e a cobrança é na escolha de ferramenta.